

# OOPs Through Python ❖❖❖ Interview Q&A

Interview Preparation Notes

OOPs Through Python — Interview Questions & Answers

## OOPs Through Python — Interview Questions & Answers

Detailed answers with keywords, Python examples, and edge cases for interview preparation.

### Q1. What is a Programming Paradigm and how is it useful?

- A **programming paradigm** is a style/approach of writing and organizing code based on certain principles — e.g., **procedural**, **object-oriented**, **functional**, **logical**, and **declarative** paradigms.
- It defines *how* a programmer thinks about structuring a solution — as a sequence of steps, as objects interacting, as mathematical functions, etc.
- Python is a **multi-paradigm language** — it supports procedural, OOP, and functional styles simultaneously.
- Usefulness: it gives a **standard methodology** to design, organize and maintain code, improving **readability**, **reusability**, and **team collaboration**.
- Choosing the right paradigm improves **scalability** and **maintainability** of large systems (e.g., OOP for modelling real-world entities, functional for data pipelines).
- It also influences **tooling**, **debugging approach**, and **design patterns** used in a project.

Example:

```
# Procedural style
def add(a, b):
    return a + b

# OOP style
class Calculator:
    def add(self, a, b):
        return a + b
```

### Q2. Difference between Procedure Oriented Programming (POP) and Object Oriented Programming (OOP)

| Aspect             | POP                         | OOP                           |
|--------------------|-----------------------------|-------------------------------|
| Focus              | Functions/procedures        | Objects & classes             |
| Data security      | Data is global, less secure | Data hidden via encapsulation |
| Approach           | Top-down                    | Bottom-up                     |
| Code reuse         | Limited (copy-paste)        | Inheritance enables reuse     |
| Real-world mapping | Poor                        | Models real-world entities    |
| Example languages  | C                           | Python, Java, C++             |

- POP divides a program into **functions**; data moves freely between functions, so there's **low data security**.
- OOP divides the program into **objects** that bundle **data (attributes)** and **behavior (methods)** together.

- POP follows a **top-down design**, while OOP follows a **bottom-up design** (build objects first, combine into a system).
- OOP provides **encapsulation, inheritance, polymorphism, abstraction** — POP has none of these.
- Code **reusability and extensibility** are much higher in OOP due to inheritance and composition.
- Python supports both, but OOP is preferred for **large, complex, and maintainable** applications.

```
# POP
def area(l, b):
    return l * b
print(area(4, 5))

# OOP
class Rectangle:
    def __init__(self, l, b):
        self.l, self.b = l, b
    def area(self):
        return self.l * self.b
print(Rectangle(4, 5).area())
```

### Q3. What is Object Oriented Programming (OOPs)?

- OOP is a programming paradigm based on the concept of “**objects**”, which contain **data (attributes/state)** and **code (methods/behavior)**.
- It is built on **four pillars: Encapsulation, Abstraction, Inheritance, Polymorphism**.
- Real-world entities (Car, Employee, BankAccount) are modeled as **classes**, and actual instances are **objects**.
- It promotes **modularity** — each object is a self-contained unit that can be developed/tested independently.
- Python implements OOP but is **not purely object-oriented** like Java — everything is an object internally (int, str, functions) but you’re not *forced* to write classes.
- Key keywords: `class`, `object`, `self`, `__init__`, inheritance (`class Child(Parent)`), `super()`.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def display(self):
        print(f"{self.name} scored {self.marks}")

s1 = Student("Ravi", 89)
s1.display()
```

### Q4. Advantages of OOPs

- **Modularity** — code organized into classes/objects, easier to manage and debug.
- **Reusability** — via **inheritance**, existing classes can be extended without rewriting code.
- **Data security** — via **encapsulation**, internal state is protected from unauthorized access using private/protected members.
- **Flexibility** — via **polymorphism**, the same interface (method name) can behave differently for different objects.
- **Real-world mapping** — objects model real-world entities, making design intuitive.
- **Maintainability & scalability** — easier to extend a system by adding new classes without breaking existing code (Open/Closed Principle).

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r ** 2

class Square(Shape):
    def __init__(self, s): self.s = s
    def area(self): return self.s ** 2
```

```
for shape in [Circle(3), Square(4)]:
    print(shape.area())    # Polymorphism in action
```

## Q5. Object and Class — with real-world example and programs

- A **class** is a **blueprint/template** that defines attributes (data) and methods (behavior) — it does not occupy memory for data until an object is created.
- An **object** is a **runtime instance** of a class — it has its **own copy of instance attributes** and can call the class's methods.
- Real-world example: “**Car**” is a class (blueprint: brand, color, engine, drive() method); “**my red Honda City**” is an object (a specific car built from that blueprint).
- Creating an object is called **instantiation**; internally Python calls `__init__()` automatically when an object is created.
- Multiple objects can be created from the same class, each with **independent state** but **shared method code**.
- Objects are stored in memory (heap), and a variable holds a **reference** to that object.

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
    def drive(self):
        print(f"{self.color} {self.brand} is driving")

car1 = Car("Honda City", "Red")
car2 = Car("Swift", "White")
car1.drive()
car2.drive()
```

## Q6. Differences between Class and Object

| Class                                          | Object                                     |
|------------------------------------------------|--------------------------------------------|
| Blueprint/template                             | Instance of a class                        |
| Logical entity, no memory for instance data    | Physical entity, occupies memory           |
| Declared once using <code>class</code> keyword | Many objects can be created from one class |
| Defines attributes & methods                   | Holds actual values for attributes         |
| Doesn't exist at runtime by itself             | Exists at runtime                          |

- A class is created only **once** using the `class` keyword; objects can be created **any number of times** from it.
- Class defines the **structure**; the object gives that structure **real data**.
- Memory is allocated **only when an object is instantiated**, not when a class is defined.
- Each object has its own **instance variables**, but they **share the same methods** defined in the class (methods live in class memory, not duplicated per object).
- You access class-level things using `ClassName.attr`; object-level things using `object.attr`.
- `type(object)` returns the class of an object, showing the parent-child relation.

```
class Dog:
    sound = "Bark"    # class attribute

d1 = Dog()
d2 = Dog()
print(type(d1))      # <class '.__main__.Dog'>
print(d1.sound, d2.sound)
```

## Q7. What is a variable and how many types of variables are there in OOPs?

- A **variable** is a named memory location used to store data/values that can change during program execution.

- In Python, variables are **references (labels/tags)** pointing to objects in memory — Python is **dynamically typed**, so no explicit type declaration is needed.
- In OOPs context, variables are broadly classified into: **Local variables**, **Instance variables**, **Class/Static variables**, and **Reference variables**.
- **Local variables** exist only inside a method/function and die once the function returns.
- **Instance variables** belong to a specific object, created using `self.varname` inside methods (usually `__init__`).
- **Class variables** are shared across all objects of a class, declared directly inside the class body (outside methods).

```
class Employee:
    company = "TCS"          # class variable

    def __init__(self, name):
        self.name = name    # instance variable

    def show(self):
        bonus = 5000        # local variable
        print(self.name, self.company, bonus)
```

## Q8. What is self variable and advantages of self variable in Python?

- `self` is the **first parameter** of every instance method; it refers to the **current object** on which the method is called.
- It is not a keyword — it's just a **convention** (you could name it anything, but `self` is the standard).
- `self` allows each object to access its **own instance variables and methods**, distinguishing it from other objects of the same class.
- Python automatically passes the calling object as the first argument — `obj.method()` internally becomes `Class.method(obj)`.
- **Advantages:** enables **object-specific data access**, supports **method chaining**, allows referring to other methods/attributes of the same object, and is essential for **encapsulation**.
- Without `self`, instance variables cannot be created/accessed inside methods — Python would not know *which* object's data to use.

```
class Person:
    def __init__(self, name):
        self.name = name      # self binds 'name' to this object
    def greet(self):
        print(f"Hello, I am {self.name}")

p1 = Person("Amit")
p2 = Person("Sara")
p1.greet()    # Hello, I am Amit
p2.greet()    # Hello, I am Sara
```

## Q9. What are Local Variables and Reference Variables?

- **Local variable:** declared **inside a method/function**; scope is limited to that block; destroyed once the function execution completes.
- **Reference variable:** a variable that **stores the memory address (reference)** of an object rather than the object itself — e.g., `p1 = Person()`, here `p1` is a reference variable.
- In Python, **all variables are essentially references** to objects (everything is an object) — assignment doesn't copy data, it copies the reference.
- Local variables are created on the **function call stack**; reference variables to objects point to data on the **heap**.
- Multiple reference variables can point to the **same object** — modifying via one reference reflects in the other (important edge case for mutable objects like lists).
- Reference variables enable **object identity checks** using `is` (compares memory address) vs `==` (compares value).

```
def calc():
    x = 10          # local variable, dies after function ends
    return x

class A: pass
```

```

a1 = A()
a2 = a1          # a2 is a reference variable pointing to same object as a1
print(a1 is a2)  # True

```

## Q10. Difference between Local and Reference Variables

| Local Variable                       | Reference Variable                                            |
|--------------------------------------|---------------------------------------------------------------|
| Declared inside function/method body | Points to/holds address of an object                          |
| Scope limited to that function       | Scope depends on where it's assigned (can be instance/global) |
| Stored on stack, temporary           | Refers to heap-allocated object                               |
| Destroyed after function returns     | Exists as long as reference is alive / reachable              |
| Holds primitive-like values directly | Holds address, not the actual object data                     |

- A local variable's **lifetime is tied to the function call**; a reference variable's lifetime is tied to **scope and garbage collection**.
- Local variables can themselves **be** reference variables if they store an object (e.g., `p = Person()` inside a method is local + reference).
- Reassigning a reference variable **does not affect** the original object; it just points the variable to a new object.
- Passing a reference variable to a function passes the **reference by value** — mutable objects can still be modified inside the function (edge case: **pass-by-object-reference** in Python).
- Deleting a reference variable with `del` removes the reference, not necessarily the object (object is destroyed only when **no references remain**, handled by garbage collector).
- Understanding this distinction is key to avoiding **aliasing bugs** with mutable objects like lists/dicts.

```

def modify(lst):
    lst.append(4)    # modifies the SAME object via reference

nums = [1, 2, 3]
modify(nums)
print(nums)        # [1, 2, 3, 4] -- reference variable effect

```

## Q11. What is an Instance Variable and how does it differ from Local Variables?

- Instance variable:** a variable **bound to an object** using `self.varname`, usually created inside `__init__()`; each object gets its **own independent copy**.
- Local variable:** declared inside any method **without** `self`; exists only during that method call and is **not accessible outside** it.
- Instance variables represent the **state of an object** and persist as long as the object exists.
- Local variables are used for **temporary computation** within a method and are discarded once the method finishes execution.
- Instance variables are accessed via `self.var` or `object.var`; local variables are accessed directly by name only within their function.
- Edge case: if a local variable has the **same name** as an instance variable, the local one **shadows** it inside that method (no automatic linking) — you must explicitly use `self.` to refer to the instance variable.

```

class Account:
    def __init__(self, balance):
        self.balance = balance    # instance variable

    def apply_interest(self):
        rate = 0.05               # local variable
        self.balance += self.balance * rate

a = Account(1000)
a.apply_interest()
print(a.balance)    # 1050.0

```

## Q12. What are Class-level (Static) Variables and how do they differ from Instance Variables?

- A **class variable (static variable)** is declared **directly inside the class body**, outside any method — it is **shared by all objects** of that class.
- Unlike instance variables, class variables are **not duplicated per object**; there's only **one copy** in memory, tied to the class itself.
- Accessed via `ClassName.var` (preferred) or `object.var` (works too, but doesn't modify the class-level copy if reassigned via instance).
- Useful for data common to all objects — e.g., **company name, counter of objects created, tax rate**.
- Edge case: `self.var = value` inside a method **creates a new instance variable** that shadows the class variable for that object only — it does **not** update the shared class variable.
- Class variables are typically modified via `ClassName.var = new_value`, affecting all objects that haven't overridden it at instance level.

```
class Employee:
    count = 0          # class/static variable
    def __init__(self, name):
        self.name = name    # instance variable
        Employee.count += 1  # updates shared class variable

e1 = Employee("A")
e2 = Employee("B")
print(Employee.count)    # 2
```

## Q13. Differences between Instance Variables and Class-level (Static) Variables

| Instance Variable                                         | Class Variable (Static)                     |
|-----------------------------------------------------------|---------------------------------------------|
| Declared with <code>self.var</code> inside methods        | Declared inside class body, outside methods |
| Separate copy for each object                             | Single copy shared by all objects           |
| Represents object-specific state                          | Represents data common to the whole class   |
| Accessed via <code>self.var</code> / <code>obj.var</code> | Accessed via <code>ClassName.var</code>     |
| Changing it affects only that object                      | Changing via class name affects all objects |
| Created at object creation time                           | Created at class definition time            |

- Instance variables model **“what is unique to this object”**; class variables model **“what is common to all objects”**.
- Memory-wise, instance variables multiply with the number of objects; class variables stay constant (**memory efficient** for shared data).
- If you do `obj.class_var = x`, Python creates a **new instance variable** `class_var` for that object — the original class variable is unaffected (common interview trap).
- Class variables are often used for **counters, constants, default configuration values**.
- Both can be accessed inside instance methods via `self`, but only class variables can be accessed via `ClassName` directly.
- Static/class variables are analogous to `static` fields in Java/C++.

```
class Counter:
    total = 0
    def __init__(self):
        Counter.total += 1
        self.id = Counter.total    # instance variable, unique per object

c1, c2 = Counter(), Counter()
print(c1.id, c2.id, Counter.total)    # 1 2 2
```

## Q14. What is a Constructor in Python?

- A **constructor** is a special method automatically invoked when an object is **created/instantiated**, used to **initialize the**

### object's state (instance variables).

- In Python, the constructor is defined using the `__init__()` dunder method.
- It is called **implicitly** — you never call `__init__()` directly (though technically you can); Python calls it right after `__new__()` creates the object.
- A constructor can take parameters (besides `self`) to initialize instance variables with values passed during object creation.
- Python supports **only one** `__init__` per class (no traditional overloading) — default arguments/`*args` / `**kwargs` are used to simulate multiple constructor signatures.
- If no `__init__` is defined, Python uses a **default constructor** that does nothing extra besides creating the object.

```
class Book:
    def __init__(self, title, price):
        self.title = title
        self.price = price

b = Book("Python 101", 499)  # __init__ called automatically
```

## Q15. Advantages of Constructor in Python

- **Automatic initialization** — object attributes are set up automatically at creation time, avoiding manual assignment after object creation.
- **Reduces code duplication** — initialization logic is written once, reused every time an object is created.
- **Ensures object validity** — you can add validation logic inside `__init__` so no object is created in an invalid/incomplete state.
- Supports **default values** for parameters, giving flexibility for optional initialization ( `def __init__(self, name, age=18)` ).
- Improves **readability and maintainability** — anyone reading the class immediately sees what data an object requires.
- Works seamlessly with **inheritance** — child classes can call parent constructor via `super().__init__()` to reuse initialization logic.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        if balance < 0:
            raise ValueError("Balance cannot be negative")
        self.owner = owner
        self.balance = balance

acc = BankAccount("Neha")  # default balance used, validated safely
```

## Q16. Explain the `__init__()` method in Python and how it works

- `__init__()` is a **dunder (double underscore) method**, also called the **initializer**, automatically called right after an object is created.
- It is **not technically the constructor** in the strict sense — `__new__()` is the actual object-creating method; `__init__()` only **initializes** the already-created object (an important nuance/edge case).
- Its first parameter is always `self`, representing the newly created instance; additional parameters initialize instance attributes.
- It **does not return any value** (implicitly returns `None`) — attempting to `return` a value from `__init__` raises a `TypeError`.
- If a subclass overrides `__init__`, the parent's `__init__` is **not called automatically** — you must explicitly call `super().__init__()` if parent initialization is needed.
- Execution order: `ClassName(args)` → Python calls `__new__()` to create the object → then calls `__init__(self, args)` to initialize it.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(3, 4)  # __new__ creates object, then __init__ sets x, y
print(p.x, p.y)
```

## Q17. Difference between Constructor and Method in Python

| Constructor ( <code>__init__</code> )            | Regular Method                                       |
|--------------------------------------------------|------------------------------------------------------|
| Called automatically at object creation          | Called explicitly using <code>object.method()</code> |
| Name is always <code>__init__</code>             | Any user-defined name                                |
| Used to initialize instance variables            | Used to define object behavior/logic                 |
| Only one <code>__init__</code> allowed per class | Multiple methods allowed                             |
| Cannot return a value (must return None)         | Can return any value                                 |
| Invoked only once per object (at creation)       | Can be invoked multiple times                        |

- A constructor's purpose is **initialization**; a method's purpose is to **perform an operation/behavior** on the object's data.
- The constructor runs **only once**, right when the object is created; methods can be called **any number of times** afterward.
- Constructors are a special case of methods (they're dunder methods) but are treated differently by the Python runtime (auto-invoked).
- Both take `self` as the first parameter to operate on instance data.
- Methods can be **overridden** in child classes just like `__init__` can be overridden.
- Constructors cannot be overloaded traditionally in Python (see Q53); regular methods face the same limitation.

```
class Circle:
    def __init__(self, r):      # constructor
        self.r = r
    def area(self):            # regular method
        return 3.14 * self.r ** 2

c = Circle(5)
print(c.area())
```

## Q18. Define Method and explain different types of methods in a class

- A **method** is a function defined **inside a class** that operates on the class's data (attributes) and is called on an object or the class itself.
- Python classes support **three types of methods**: **Instance methods**, **Class methods**, and **Static methods**.
- **Instance methods**: take `self` as the first parameter, operate on instance-specific data, called via `object.method()`.
- **Class methods**: take `cls` as the first parameter, decorated with `@classmethod`, operate on class-level data, called via `ClassName.method()`.
- **Static methods**: take no `self` / `cls`, decorated with `@staticmethod`, behave like a plain function placed inside the class for logical grouping/namespacing.
- Choice of method type depends on **what data it needs to access**: instance data → instance method; class data → class method; no class/instance data → static method.

```
class Demo:
    count = 0

    def instance_method(self):
        print("Instance method, self =", self)

    @classmethod
    def class_method(cls):
        print("Class method, cls =", cls)

    @staticmethod
    def static_method():
        print("Static method, no self/cls")

d = Demo()
d.instance_method()
```

```
Demo.class_method()
Demo.static_method()
```

## Q19. What is a Non-static (Instance) Method? — with example program

- An **instance method** is the most common type of method; it operates on **instance-specific data** and requires an **object** to be called (though it can technically be called via `ClassName.method(obj)` ).
- Its first parameter is always `self` , which Python automatically binds to the calling object.
- It can access and modify **both instance variables and class variables** through `self` and `ClassName` .
- It is used when the method's logic **depends on the object's own state** (e.g., calculating something using `self.attr` ).
- Instance methods support **polymorphism and overriding**, since subclasses can redefine them for different behavior.
- Edge case: if called on the class directly ( `ClassName.method()` ), Python raises `TypeError` because `self` is not automatically supplied — you'd need `ClassName.method(instance)` .

```
class Employee:
    def __init__(self, name, salary):
        self.name, self.salary = name, salary

    def give_raise(self, amount):  # instance method
        self.salary += amount
        print(f"{self.name}'s new salary: {self.salary}")

e = Employee("Kiran", 30000)
e.give_raise(5000)
```

## Q20. What is a Static Method / Class Method? — with example program

- A **static method** ( `@staticmethod` ) does not receive `self` or `cls` automatically — it behaves like a **regular function** but is logically grouped inside the class's namespace.
- A **class method** ( `@classmethod` ) receives `cls` (the class itself) as its first parameter, allowing it to access/modify **class-level (static) variables** regardless of which object calls it.
- Static methods are used for **utility/helper functions** related to the class but not dependent on instance or class state (e.g., validation, math helpers).
- Class methods are commonly used as **factory methods** — alternate ways to construct objects ( `@classmethod def from_string(cls, data): ...` ).
- Both are called using either `ClassName.method()` or `object.method()` , but conceptually meant to be called on the **class**.
- Edge case: class methods automatically work correctly with **inheritance** — `cls` refers to the actual subclass that called the method, not always the base class (useful for factory patterns).

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b  # no self/cls needed

class Employee:
    count = 0
    def __init__(self, name):
        self.name = name
        Employee.count += 1

    @classmethod
    def total_employees(cls):
        return cls.count  # accesses class variable via cls

print(MathUtils.add(3, 4))
Employee("A"); Employee("B")
print(Employee.total_employees())  # 2
```

## Q21. What is a “named” Decorator in Python and what is its advantage?

- In Python, a decorator is referred to by its **name** using the `@decorator_name` syntax placed directly above a function/method definition — this is called **decorator (named) syntax / syntactic sugar**.
- It is equivalent to writing `function = decorator_name(function)` manually, but the `@name` form is more **readable and declarative**.
- Named decorators like `@staticmethod`, `@classmethod`, `@property`, `@abstractmethod` are **built-in** and instantly communicate the method's role to any reader.
- Advantage: it **separates cross-cutting concerns** (logging, validation, access control, memoization) from the core logic, without modifying the original function's code.
- Custom named decorators can be created as regular functions that accept a function and return a wrapped version of it.
- Advantage summary: **cleaner syntax, reusability, separation of concerns, and improved readability** compared to manually wrapping functions.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before function call")
        result = func(*args, **kwargs)
        print("After function call")
        return result
    return wrapper

@my_decorator          # named decorator syntax
def greet(name):
    print(f"Hello {name}")

greet("Ravi")
```

## Q22. Difference between Method and Constructor in Python OOPs

(See also Q17 — same core comparison, restated with emphasis on OOP role)

- **Purpose:** Constructor **initializes** object state; method **implements behavior/operations**.
- **Invocation:** Constructor is invoked **automatically** once, at object creation; methods are invoked **explicitly**, any number of times.
- **Naming:** Constructor always has the fixed name `__init__`; methods can have any valid identifier as a name.
- **Return type:** Constructor implicitly returns `None` only; methods can return any data type.
- **Overloading:** Neither supports true overloading in Python — both rely on default args/ `*args` to simulate it.
- **Relationship with inheritance:** constructors need `super().__init__()` to chain parent initialization; methods use `super().method()` to reuse/extend parent behavior.

```
class Vehicle:
    def __init__(self, brand):    # constructor
        self.brand = brand
    def start(self):             # method
        print(f"{self.brand} started")
```

## Q23. What is Data Encapsulation? — real-world examples in Python

- **Encapsulation** is the OOP principle of **bundling data (attributes) and methods** that operate on that data into a single unit (class), while **restricting direct access** to some of the object's components.
- It is achieved in Python using **access modifiers**: public (default), protected (`_var`), and private (`__var`), combined with **getter/setter methods**.
- Real-world example 1: A **capsule/medicine** — the outer coating hides the inner chemical composition; you consume it through a defined interface (swallowing), not by breaking it open.
- Real-world example 2: An **ATM machine** — internal banking logic and account data are hidden; the user interacts only via defined buttons/interfaces (withdraw, deposit).
- Real-world example 3: A **car** — the engine's internal mechanism is hidden from the driver; the driver uses the steering wheel and pedals (public interface) to control it.
- Advantage: encapsulation provides **data security, controlled access, reduced complexity**, and protects object state from **accidental/unauthorized modification**.

```

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private attribute (data hiding)

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):           # controlled access via getter
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
print(acc.get_balance())    # 1500

```

## Q24. What is Data Hiding and how can we achieve it in Python?

- **Data hiding** is the practice of **restricting direct access** to an object's internal data from outside the class, exposing it only through controlled methods (getters/setters).
- It is a **subset/mechanism of encapsulation** — encapsulation is the concept, data hiding is the implementation technique.
- Achieved in Python by prefixing attribute names with **double underscore** `__` (e.g., `self.__salary`), which triggers **name mangling**.
- Python does **not have true private members** like Java/C++ — data hiding is achieved by **convention + name mangling**, not by strict compiler enforcement (important edge case).
- Single underscore `_var` indicates **"protected" by convention** (accessible, but signals "internal use only" to developers) — not enforced by Python either.
- Access to hidden/private data should be provided through **public methods (getters/setters)** or the `@property` decorator, keeping validation logic centralized.

```

class Employee:
    def __init__(self, salary):
        self.__salary = salary    # name-mangled to _Employee__salary

    def get_salary(self):
        return self.__salary

e = Employee(50000)
# print(e.__salary)    # AttributeError
print(e._Employee__salary)    # 50000 -- still technically accessible (not truly private)

```

## Q25. What is an Access Modifier and its different types?

- An **access modifier** controls the **visibility/accessibility** of a class's attributes and methods from outside the class (or from subclasses).
- Python provides **three levels by convention: Public, Protected, and Private** — unlike Java/C++, these are **not enforced by the compiler**, only by naming convention.
- **Public** members: no underscore prefix, accessible from anywhere (`self.name`).
- **Protected** members: single underscore prefix `_name`, meant for internal/subclass use, but still accessible from outside (convention only).
- **Private** members: double underscore prefix `__name`, subjected to **name mangling** (`_ClassName__name`), making accidental access harder but not impossible.
- Understanding these is essential for good **encapsulation design** even though Python trusts the developer ("we're all consenting adults" philosophy).

```

class Demo:
    def __init__(self):
        self.public_var = 1    # accessible anywhere
        self._protected_var = 2    # convention: internal/subclass use
        self.__private_var = 3    # name-mangled

d = Demo()

```

```
print(d.public_var, d._protected_var)
```

## Q26. Public, Protected and Private Access Specifiers in Python

- **Public** ( `self.var` ): default access; can be accessed and modified from **anywhere** — inside class, subclass, or outside code.
- **Protected** ( `self._var` ): single underscore convention indicating the member is meant for use **within the class and its subclasses only** — Python does **not** technically block outside access.
- **Private** ( `self.__var` ): double underscore triggers **name mangling** to `_ClassName__var`, making it harder (not impossible) to access accidentally from outside the class.
- Protected members are typically used when a class is designed to be **inherited**, signaling “override or use carefully in subclasses.”
- Private members are used to **truly hide implementation details**, exposing behavior only through public methods.
- Edge case: name mangling applies even **within the class hierarchy** — a subclass cannot directly access a parent’s `__private` attribute by the same name; it must use the parent’s public/protected methods.

```
class Parent:
    def __init__(self):
        self.pub = "public"
        self._prot = "protected"
        self.__priv = "private"

class Child(Parent):
    def show(self):
        print(self.pub)           # OK
        print(self._prot)        # OK (convention)
        # print(self.__priv)     # AttributeError - name mangled to _Parent__priv
```

## Q27. Differences between Private and Public Access Specifiers in Python

| Public                                     | Private                                                    |
|--------------------------------------------|------------------------------------------------------------|
| No underscore prefix                       | Double underscore <code>__</code> prefix                   |
| Accessible from anywhere                   | Access restricted via name mangling                        |
| No mangling applied                        | Name mangled to <code>_ClassName__var</code>               |
| Default choice for most attributes/methods | Used to hide sensitive/internal implementation             |
| Easily inherited and overridden            | Not directly inherited by name in subclasses               |
| No extra syntax needed to access           | Needs mangled name or getter method to access from outside |

- Public members form the **class’s external interface** — meant to be used by any client code.
- Private members form the **internal implementation details** — meant to be hidden and accessed only via defined public methods.
- Public attributes can be freely reassigned from outside; private attributes require a **setter method** for controlled reassignment (good practice).
- Private variables help prevent **accidental overwrites** in large codebases and complex inheritance chains.
- Public methods typically act as the **API** of the class; private methods ( `def __helper(self)` ) are internal helpers not meant for external use.
- Both are **conventions in Python**, not strictly enforced like access modifiers in Java ( `private` keyword ) or C++.

```
class Wallet:
    def __init__(self, amount):
        self.__amount = amount    # private
        self.currency = "INR"     # public

    def show_balance(self):
        return f"{self.__amount} {self.currency}"
```

## Q28. Differences between Static and Non-static (Instance) Methods in Python

| Static Method                                                           | Instance (Non-static) Method                             |
|-------------------------------------------------------------------------|----------------------------------------------------------|
| Decorated with <code>@staticmethod</code>                               | No special decorator needed                              |
| No <code>self</code> / <code>cls</code> parameter                       | First parameter is <code>self</code>                     |
| Cannot access instance or class data directly                           | Can access instance and class data via <code>self</code> |
| Called via <code>ClassName.method()</code> or <code>obj.method()</code> | Called via <code>obj.method()</code>                     |
| Behaves like a plain function inside class namespace                    | Bound to a specific object instance                      |
| Used for utility/helper logic                                           | Used for logic dependent on object's state               |

- Static methods are used when the method's logic is **related to the class conceptually** but doesn't need any object or class data (e.g., a validation utility).
- Instance methods are the **default and most common** method type, needed whenever object-specific data (`self.attr`) must be read/modified.
- Static methods **cannot call other instance methods** directly (no `self` reference) — they'd need an explicit object reference passed in.
- Instance methods automatically support **polymorphism/overriding**; static methods can be overridden too but don't participate in dynamic dispatch based on instance type in the same way.
- Static methods improve **namespace organization** — grouping related utility functions logically under a class instead of as loose module-level functions.
- Choosing wrongly (e.g., making everything static) breaks proper OOP design and loses the benefit of encapsulated, object-specific behavior.

```
class TemperatureConverter:
    @staticmethod
    def c_to_f(c):                # static: no self needed
        return (c * 9/5) + 32

    def __init__(self, city, celsius):
        self.city = city
        self.celsius = celsius

    def report(self):             # instance: needs self.celsius
        f = TemperatureConverter.c_to_f(self.celsius)
        print(f"{self.city}: {f} F")
```

## Q29. Wrapper Function in Decorators — with example program

- A **wrapper function** is an **inner function defined inside a decorator**, which “wraps” (surrounds) the original function's call with additional logic — code **before and/or after** the actual call.
- The decorator function receives the original function as an argument and **returns the wrapper function**, replacing the original in the calling namespace.
- The wrapper typically accepts `*args, **kwargs` to accept **any number/type of arguments**, making the decorator generic and reusable for any function signature.
- It enables adding behavior like **logging, timing, authentication, caching** without modifying the original function's source code — following the **Open/Closed Principle**.
- Multiple decorators can be **stacked**, and each wrapper calls the next inner function/wrapper, forming a **chain of wrapping calls**.
- Edge case: without using `functools.wraps`, the wrapped function **loses its original metadata** (`__name__`, `__doc__`), which can confuse debugging tools — `functools.wraps(func)` should be applied to the wrapper.

```
import functools

def log_calls(func):
    @functools.wraps(func)        # preserves original metadata
    def wrapper(*args, **kwargs): # wrapper function
        print(f"Calling {func.__name__} with {args}")
        result = func(*args, **kwargs)
        return result
```

```

        print(f"{func.__name__} returned {result}")
    return result
return wrapper

@log_calls
def multiply(a, b):
    return a * b

multiply(3, 4)

```

### Q30. What is Name Mangling in Python and its advantages?

- **Name mangling** is a mechanism where Python internally renames any identifier of the form `__name` (double leading underscore, at most one trailing underscore) inside a class to `__ClassName__name`.
- It happens automatically at **compile time**, purely as a **textual transformation** — it is not true access control/security.
- Its main purpose is to **avoid naming conflicts** in inheritance hierarchies — a subclass accidentally defining the same `__var` name as its parent won't clash, because each gets mangled with its own class name.
- Advantage 1: Prevents **accidental overriding** of “private” attributes/methods when a subclass unintentionally reuses the same variable name.
- Advantage 2: Provides a **soft form of data hiding**, discouraging (though not strictly blocking) direct external access.
- Edge case: name mangling only applies to identifiers used **inside class body code**, and only to names with **at most one trailing underscore** (`__name` mangled, but `__name__` — dunder — is NOT mangled, since it's reserved for special/magic methods).

```

class Base:
    def __init__(self):
        self.__data = "base data"    # mangled to __Base__data

class Derived(Base):
    def __init__(self):
        super().__init__()
        self.__data = "derived data" # mangled to __Derived__data (no clash!)

d = Derived()
print(d.__Base__data, d.__Derived__data)    # both exist independently

```

### Q31. Explain the `dir()` function in Python

- `dir()` is a **built-in introspection function** that returns a **sorted list of names (attributes and methods)** valid in the current scope, or defined for a given object/class/module.
- Called without arguments, `dir()` returns names in the **current local scope**.
- Called with an object/class (`dir(obj)`), it returns **all attributes, methods, and inherited members**, including dunder methods (`__init__`, `__str__`, etc.).
- It's extremely useful for **debugging, exploring unfamiliar libraries/objects**, and understanding what an object can do (interactive introspection).
- `dir()` internally uses the object's `__dir__()` method if defined, otherwise it inspects `__dict__`, class hierarchy, and built-in attributes.
- It does not show **values**, only **names** — combine with `getattr()` to fetch actual attribute values dynamically.

```

class Student:
    def __init__(self, name):
        self.name = name
    def greet(self):
        print("Hi")

s = Student("Ravi")
print(dir(s))
# includes: ['__class__', '__init__', ..., 'greet', 'name']

```

### Q32. What is a Decorator and its advantages?

- A **decorator** is a function that takes another function (or class) as input and **extends/modifies its behavior without changing its actual source code**.
- Implemented using the `@decorator_name` syntax placed above a function/method definition.
- Python decorators leverage the fact that **functions are first-class objects** — they can be passed around, returned, and assigned like any other value.
- **Advantages:** promotes **code reusability** (write cross-cutting logic once, apply to many functions), keeps functions **clean and focused** (Single Responsibility Principle), and allows **dynamic behavior modification** at runtime.
- Common built-in decorators: `@staticmethod`, `@classmethod`, `@property`, `@abstractmethod`, `functools.lru_cache`.
- Custom decorators are widely used for **logging, authentication/authorization, caching, timing, retry logic, and validation** in real-world applications.

```
def require_login(func):
    def wrapper(user, *args, **kwargs):
        if not user.get("logged_in"):
            print("Access denied")
            return
        return func(user, *args, **kwargs)
    return wrapper

@require_login
def view_profile(user):
    print(f"Welcome {user['name']}")

view_profile({"name": "Ravi", "logged_in": True})
```

### Q33. Different types of built-in Decorators in Python

#### i) `@classmethod`

- Marks a method to receive the **class ( cls )** as its first argument instead of an object.
- Used for **factory methods** and operations on **class-level (static) data**.

```
class Pizza:
    def __init__(self, size): self.size = size
    @classmethod
    def large(cls): return cls("Large")
```

#### ii) `@staticmethod`

- Marks a method that needs **no self or cls** — behaves like a plain function grouped inside the class.
- Used for **utility/helper logic** related conceptually to the class.

```
class MathOps:
    @staticmethod
    def square(x): return x * x
```

#### iii) `@abstractmethod`

- From the `abc` module; marks a method that **must be overridden** by any concrete subclass.
- The class containing it must inherit from `ABC` (or use `ABCMeta`); such a class **cannot be instantiated directly**.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
```

#### iv) `@dispatch` (from `multipledispatch` module)

- Enables **function/method overloading based on argument types/count**, which Python doesn't support natively.
- Requires installing the third-party `multipledispatch` package ( `pip install multipledispatch` ).

```
from multipledispatch import dispatch
class Calc:
    @dispatch(int, int)
    def add(self, a, b): return a + b
```

```
@dispatch(int, int, int)
def add(self, a, b, c): return a + b + c
```

- **Common advantage across all:** they modify/extend method behavior **declaratively**, keeping the class definition clean, readable, and following DRY (Don't Repeat Yourself) principles.
- Edge case: `@abstractmethod` alone does **not prevent instantiation** unless the class also inherits `ABC` /uses `ABCMeta` — a common interview trap.

### Q34. What are Magic Methods (Dunder Methods) in Python?

- **Magic methods** (also called **dunder methods**, short for “double underscore”) are special predefined methods surrounded by double underscores, e.g., `__init__`, `__str__`, `__add__`, `__len__`.
- They allow **custom classes to integrate with Python's built-in syntax and operators** — e.g., defining `__add__` lets objects use the `+` operator.
- Python calls these methods **implicitly** in response to certain operations (`len(obj)` → calls `obj.__len__()`; `print(obj)` → calls `obj.__str__()`).
- Common categories: **object lifecycle** (`__init__`, `__new__`, `__del__`), **representation** (`__str__`, `__repr__`), **operator overloading** (`__add__`, `__sub__`, `__eq__`, `__lt__`), **container behavior** (`__len__`, `__getitem__`, `__setitem__`, `__contains__`), and **context managers** (`__enter__`, `__exit__`).
- They are the foundation of **operator overloading** and enabling custom objects to behave like built-in types.
- Edge case: `__repr__` should return an **unambiguous, developer-facing** string (ideally recreatable code); `__str__` returns a **user-friendly, readable** string — if `__str__` is missing, Python falls back to `__repr__`.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __str__(self):
        return f"Point({self.x}, {self.y})"
    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

p1, p2 = Point(1, 2), Point(3, 4)
print(p1 + p2) # calls __add__, then __str__ -> Point(4, 6)
```

### Q35. What is Data Abstraction in Python? — real-world example

- **Abstraction** is the OOP principle of **hiding internal implementation details** and exposing only the **essential features/interface** to the user.
- It focuses on “**what an object does**” rather than “**how it does it**”.
- Real-world example: driving a **car** — you use the steering wheel, accelerator, and brake (interface) without knowing the internal combustion/engine mechanics (implementation).
- Real-world example: an **ATM** — the user sees “Withdraw,” “Deposit,” “Check Balance” options, but the actual banking logic/database operations are hidden.
- Abstraction reduces **complexity** for the user of a class and improves **security** by hiding sensitive implementation logic.
- Difference from encapsulation: encapsulation is about **binding data + restricting access** (the “how”), abstraction is about **hiding complexity and showing only relevant features** (the “what”) — they're related but conceptually distinct.

```
class CoffeeMachine:
    def __brew_process(self): # hidden implementation
        print("Heating water, grinding beans...")
    def make_coffee(self): # exposed interface
        self.__brew_process()
        print("Coffee ready!")

CoffeeMachine().make_coffee() # user doesn't see internal steps directly
```

### Q36. How do we achieve Data Abstraction in Python?

- **Abstract classes and abstract methods** using the `abc` module (`ABC`, `@abstractmethod`) — forcing subclasses to implement specific behavior while hiding “how” at the base class level.

- **Access modifiers** ( `_protected` , `__private` ) — hiding internal attributes/methods while exposing only public methods as the interface.
- **Interfaces via duck typing** — Python doesn't have formal interfaces, but abstract base classes serve a similar purpose, defining a **contract** subclasses must follow.
- Using **properties** ( `@property` ) to expose a controlled, simplified interface to internally complex computed values.
- Grouping complex logic inside **private helper methods** ( `__helper()` ) and exposing only a simple **public method** as the entry point.
- Abstraction is a **design-time concept** — Python enforces it partially (via `abc` module for abstract classes) but largely relies on **developer discipline and convention**.

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def process_payment(self, amount):
        pass          # "what" is defined, "how" is hidden

class UpiPayment(PaymentGateway):
    def process_payment(self, amount):
        print(f"Processing UPI payment of {amount}")

UpiPayment().process_payment(500)
```

### Q37. What is an Abstract Method?

- An **abstract method** is a method that is **declared but not implemented** in the base/abstract class — it only defines a **method signature (contract)**.
- Declared using the `@abstractmethod` decorator from the `abc` module, inside a class that inherits from `ABC`.
- Any **concrete subclass must override and implement** the abstract method; otherwise, the subclass also becomes abstract and cannot be instantiated.
- It enforces a **common interface** across multiple subclasses, ensuring consistency (e.g., every `Shape` subclass must implement `area()` ).
- An abstract method's body can still contain code (e.g., `pass` or even default logic), but subclasses are still **required** to override it if the class is defined properly.
- Edge case: simply naming a method with `@abstractmethod` in a normal class (not inheriting `ABC` / `ABCMeta` ) does **not** enforce anything — Python will happily instantiate it without error.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r ** 2

# shape = Shape()          # TypeError: Can't instantiate abstract class
c = Circle(5)
print(c.area())
```

### Q38. What is an Abstract Class? — advantages

- An **abstract class** is a class that **cannot be instantiated directly**; it serves as a **base template** containing one or more abstract methods that subclasses must implement.
- Created in Python by inheriting from `ABC` (Abstract Base Class) provided by the `abc` module, or by setting `metaclass=ABCMeta` .
- It can contain a **mix** of abstract methods (no implementation) and **concrete methods** (fully implemented, inherited as-is by subclasses).
- **Advantages:** enforces a **consistent interface/contract** across all subclasses, prevents accidental instantiation of

incomplete classes, supports **polymorphism** by guaranteeing certain methods exist, and improves **code design and maintainability** for large systems/frameworks.

- Abstract classes are commonly used in **framework/library design** — defining a template that plugin developers must follow.
- Edge case: an abstract class **can have a constructor** ( `__init__` ) with real logic; subclasses can call it via `super().__init__()` even though the parent itself can't be instantiated directly.

```
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, name):
        self.name = name
    @abstractmethod
    def calculate_salary(self):
        pass
    def show(self):
        # concrete method
        print(f"Employee: {self.name}")

class Manager(Employee):
    def calculate_salary(self):
        return 80000

m = Manager("Priya")
m.show()
print(m.calculate_salary())
```

### Q39. What is the drawback of Abstract Classes in Python?

- **Single inheritance restriction feel:** since abstract classes are usually meant to be the “base contract,” heavy reliance on them can push designs toward rigid hierarchies, making later restructuring difficult.
- **Cannot be instantiated,** so extra boilerplate (a concrete subclass) is always required even for quick testing/prototyping.
- Adds **complexity and overhead** for small/simple programs where plain classes or duck typing would be simpler and sufficient (Python favors “duck typing” — abstract classes can feel over-engineered).
- If a **new abstract method is added later** to the base class, **all existing subclasses break** (they become abstract too) until updated — a maintenance burden in large codebases.
- Python’s abstract class enforcement is **only checked at instantiation time**, not at class-definition time — errors surface **late**, only when someone tries to create an object of an incomplete subclass.
- Multiple inheritance with several abstract classes can lead to **complex MRO (Method Resolution Order) issues**, making the design harder to reason about.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self): pass
    @abstractmethod
    def perimeter(self): pass # added later -> breaks old subclasses missing this

class OldCircle(Shape):
    def area(self): return 3.14
    # OldCircle() # TypeError: missing perimeter() implementation
```

### Q40. Difference between Abstract Method and Concrete Method

| Abstract Method                                  | Concrete Method                                  |
|--------------------------------------------------|--------------------------------------------------|
| Declared with <code>@abstractmethod</code>       | No special decorator required                    |
| No (or placeholder) implementation in base class | Fully implemented with working logic             |
| Must be overridden by subclasses                 | Can be inherited as-is, or optionally overridden |
| Exists only meaningfully within abstract classes | Can exist in any class                           |
| Defines “what” must be done                      | Defines “how” it’s actually done                 |

Cannot be called meaningfully on its own

Can be called directly and executed

- Abstract methods act as a **contract/placeholder**; concrete methods provide the **actual working behavior**.
- A class can mix both — abstract methods force customization, concrete methods provide shared/reusable logic.
- Concrete methods in an abstract base class can still call abstract methods internally (a common design pattern — **Template Method Pattern**).
- Subclasses **must** override every abstract method to become instantiable; overriding concrete methods is **optional**.
- Abstract methods enforce **compile-time-like (instantiation-time) checking** for required behavior; concrete methods have no such enforcement.
- Both support **polymorphism**, but abstract methods guarantee that polymorphic calls will always find a real implementation in any concrete subclass.

```
from abc import ABC, abstractmethod

class Report(ABC):
    @abstractmethod
    def generate(self):          # abstract
        pass
    def print_header(self):      # concrete
        print("---- Report ----")
```

#### Q41. How do we create an Abstract Class in Python?

- **Step 1:** Import `ABC` and `abstractmethod` from the built-in `abc` module.
- **Step 2:** Define the class inheriting from `ABC` — `class MyAbstract(ABC):`.
- **Step 3:** Mark one or more methods with the `@abstractmethod` decorator, leaving their body as `pass` (or a default implementation).
- **Step 4:** Any subclass must implement **all** abstract methods to be instantiable; if not, it also stays abstract.
- Alternative approach: use `class MyAbstract(metaclass=ABCMeta)` directly, without inheriting `ABC` (both achieve the same result — `ABC` is just a convenience base class using `ABCMeta`).
- Trying to instantiate the abstract class directly raises `TypeError: Can't instantiate abstract class ... with abstract methods ...`.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start_engine(self):
        pass

class Car(Vehicle):
    def start_engine(self):
        print("Car engine started")

# v = Vehicle()      # TypeError
c = Car()
c.start_engine()
```

#### Q42. Difference between Class and Abstract Class in Python

| Regular Class                                   | Abstract Class                                                   |
|-------------------------------------------------|------------------------------------------------------------------|
| Can be instantiated directly                    | Cannot be instantiated directly                                  |
| No requirement to inherit from <code>ABC</code> | Must inherit <code>ABC</code> / use <code>ABCMeta</code>         |
| All methods usually fully implemented           | Can contain one or more abstract (unimplemented) methods         |
| No enforced contract for subclasses             | Enforces a contract — subclasses must implement abstract methods |
| Used for direct object creation                 | Used as a base/template for other classes                        |

No import required

Requires `from abc import ABC, abstractmethod`

- A regular class is **self-sufficient**; an abstract class is **intentionally incomplete**, designed to be extended.
- Abstract classes are used when you want to **enforce a common structure/interface** across multiple related subclasses.
- Regular classes support inheritance too, but there's **no compiler/runtime enforcement** that subclasses override any particular method.
- Abstract classes can still have **constructors, concrete methods, and class variables** just like regular classes.
- Design-wise, abstract classes represent an **"is-a" relationship with a mandatory contract**, whereas regular classes represent a straightforward blueprint.
- Both participate normally in Python's **inheritance and polymorphism mechanisms**.

### Q43. Advantages of Abstract Classes in Python

- Enforces **consistency** — guarantees every subclass implements certain required methods, avoiding runtime `AttributeError`s from missing methods.
- Supports **polymorphism reliably** — code that works with the abstract base type can trust that key methods exist in any subclass, regardless of which concrete class is actually used.
- Encourages good **software design** by separating the **interface (what)** from the **implementation (how)**, useful in frameworks and plugin architectures.
- **Prevents accidental instantiation** of incomplete/partial objects that don't make sense on their own (e.g., a generic `Shape` without a defined `area()`).
- Enables **code reuse** through concrete/shared methods defined in the abstract base, while still customizing the abstract parts per subclass.
- Improves **team collaboration** — the abstract class acts as documentation/contract for what each subclass developer must implement.

```
from abc import ABC, abstractmethod

class Notification(ABC):
    @abstractmethod
    def send(self, message):
        pass

class EmailNotification(Notification):
    def send(self, message):
        print(f"Email sent: {message}")

class SMSNotification(Notification):
    def send(self, message):
        print(f"SMS sent: {message}")

for n in [EmailNotification(), SMSNotification()]:
    n.send("Hello!")  # polymorphism guaranteed by the abstract contract
```

### Q44. What is Inheritance in Python? Advantages of Inheritance

- **Inheritance** is the OOP mechanism where a **new class (child/derived/subclass)** acquires the properties (attributes) and behaviors (methods) of an **existing class (parent/base/super class)**.
- Syntax: `class Child(Parent):` — the child class automatically has access to all **public and protected** members of the parent.
- It models an **"is-a" relationship** — e.g., `Dog is-a Animal`, `Manager is-a Employee`.
- **Advantages**: promotes **code reusability** (no need to rewrite common logic), supports **method overriding** for specialized behavior, enables **polymorphism** (treating different subclasses uniformly via a common parent type), and improves **maintainability** (changes in shared logic propagate to all children).
- Child classes can **add new attributes/methods** or **override** existing parent methods to customize/extend behavior.
- The child class constructor can call the parent's constructor using `super().__init__()` to reuse initialization logic.

```
class Animal:
    def __init__(self, name):
        self.name = name
```

```

def sound(self):
    print("Some generic sound")

class Dog(Animal):
    # inheritance
    def sound(self):
        # overriding
        print(f"{self.name} says Woof!")

Dog("Tommy").sound()

```

## Q45. Parent/Super/Base Class and Child/Sub/Derived Class in Python

- **Parent class** (also called **Super class** or **Base class**): the class **being inherited from**; contains common/general attributes and methods.
- **Child class** (also called **Sub class** or **Derived class**): the class that **inherits** from the parent; specializes or extends the parent's functionality.
- The relationship is defined syntactically as: `class Child(Parent):`.
- A child class automatically gets access to the parent's **public and protected** members without redefining them.
- A child class can **override** parent methods (redefine with same name) or **extend** them using `super().method()` to call the parent's version and add more logic.
- One parent can have **multiple children** (one-to-many), and Python also supports a child inheriting from **multiple parents** (multiple inheritance).

```

class Person:
    # parent/base/super class
    def __init__(self, name):
        self.name = name
    def introduce(self):
        print(f"I am {self.name}")

class Student(Person):
    # child/sub/derived class
    def __init__(self, name, roll):
        super().__init__(name) # reuse parent constructor
        self.roll = roll
    def introduce(self):
        super().introduce() # extend parent method
        print(f"My roll number is {self.roll}")

Student("Anu", 12).introduce()

```

## Q46. Different types of Inheritance in Python

- **Single Inheritance**: one child inherits from **one** parent class.
- **Multilevel Inheritance**: a chain — a class inherits from a child, which itself inherited from another parent (grandparent → parent → child).
- **Hierarchical Inheritance**: **multiple child classes** inherit from the **same single parent** class.
- **Multiple Inheritance**: a **single child class** inherits from **more than one parent** class simultaneously.
- **Hybrid Inheritance**: a **combination** of two or more of the above types in a single design (e.g., hierarchical + multiple).
- Python is one of the few mainstream OOP languages that **natively supports multiple inheritance** (unlike Java, which requires interfaces instead); this is managed internally via the **MRO (Method Resolution Order)**.

## Q47. Single, Multilevel, Hierarchical, Multiple and Hybrid Inheritance — with example programs

**Single Inheritance** — one parent, one child:

```

class Animal:
    def eat(self): print("Eating")
class Dog(Animal):
    def bark(self): print("Barking")
Dog().eat(); Dog().bark()

```

**Multilevel Inheritance** — chain of inheritance:

```
class Animal:
    def eat(self): print("Eating")
class Mammal(Animal):
    def walk(self): print("Walking")
class Dog(Mammal):
    def bark(self): print("Barking")
Dog().eat(); Dog().walk(); Dog().bark()
```

**Hierarchical Inheritance** — multiple children, one parent:

```
class Animal:
    def eat(self): print("Eating")
class Dog(Animal):
    def bark(self): print("Bark")
class Cat(Animal):
    def meow(self): print("Meow")
```

**Multiple Inheritance** — one child, multiple parents:

```
class Father:
    def skills(self): print("Driving")
class Mother:
    def skills(self): print("Cooking")
class Child(Father, Mother): # multiple inheritance
    pass
Child().skills() # uses MRO -> Father's skills() (left-to-right)
```

**Hybrid Inheritance** — combination of multiple types:

```
class Animal: # base
    def eat(self): print("Eating")
class Mammal(Animal): # hierarchical + multilevel
    def walk(self): print("Walking")
class Bird(Animal):
    def fly(self): print("Flying")
class Bat(Mammal, Bird): # multiple inheritance -> hybrid overall
    pass
Bat().eat(); Bat().walk(); Bat().fly()
```

- Each type solves a different real-world modelling need — choose based on the **actual relationships** between entities, not arbitrarily, since deep/complex hierarchies increase **maintenance difficulty**.

## Q48. MRO (Method Resolution Order) in Python — and its advantage

- **MRO** is the **order in which Python looks up methods/attributes** across a class hierarchy, especially important in **multiple inheritance** scenarios.
- Python uses the **C3 Linearization algorithm** to compute MRO, ensuring a **consistent, predictable, and monotonic** order.
- You can view a class's MRO using `ClassName.__mro__` or `ClassName.mro()`.
- **Advantage:** it resolves the **"Diamond Problem"** (ambiguity when multiple parent classes define the same method) in a deterministic way, so `super()` calls always follow a clear, single, consistent path.
- MRO ensures a class always appears **before its parents**, and if a class inherits from multiple classes, they are checked in the **order specified** at the class definition, without duplication.
- It underlies how `super()` works in cooperative multiple inheritance — `super()` doesn't just go to "the parent," it goes to the **next class in the MRO chain**.

```
class A:
    def show(self): print("A")
class B(A):
    def show(self): print("B")
class C(A):
    def show(self): print("C")
class D(B, C):
```

```

pass

print(D.__mro__)
# (D, B, C, A, object) -- C3 linearization
D().show()    # "B" -- first match found in MRO order

```

## Q49. Fundamental Rules of MRO (Two Important Rules)

1. **Local precedence order (monotonicity)**: a class must always appear **before its parent classes** in the MRO — a child's own method always takes priority over any inherited one from its ancestors.
2. **Consistent extended precedence order (order preserved as declared)**: if a class inherits from multiple parents `class D(B, C):`, then in the MRO, **B must appear before C** — the order of parents as **written in the class definition is preserved** across the whole hierarchy.
  - These two rules combined are what the **C3 Linearization algorithm** guarantees — it merges the MROs of all parent classes plus the list of parents themselves, respecting both **local precedence** and **declaration order**.
  - If these rules **cannot be satisfied simultaneously** (a genuine ordering conflict, e.g., contradictory parent orders across an inheritance graph), Python raises a **TypeError: Cannot create a consistent MRO** at class-definition time.
  - These rules make **multiple inheritance predictable and safe**, avoiding the ambiguity found in some other languages (like C++'s diamond problem issues).
  - Advantage: developers can **reliably predict** which method will execute when there are naming conflicts across multiple base classes, just by reading `ClassName.__mro__`.

```

class X: pass
class Y: pass
class A(X, Y): pass
class B(Y, X): pass
# class C(A, B): pass    # TypeError: conflicting order of X and Y -> violates rule 2

```

## Q50. Does Python support Multiple Inheritance? If yes, how?

- **Yes**, Python natively supports **multiple inheritance** — a class can inherit from **more than one parent class** simultaneously.
- Syntax: `class Child(Parent1, Parent2, ...):` — the child gains access to attributes/methods of **all** listed parent classes.
- Python resolves potential conflicts (e.g., same method name in multiple parents) using the **MRO (C3 Linearization algorithm)** — see Q48/Q49.
- Method lookup follows the computed MRO order — the **first matching implementation found (left to right, depth-first, respecting C3 rules)** is used.
- `super()` in a multiple-inheritance context follows **cooperative multiple inheritance** — it moves to the “next” class in the MRO chain, not necessarily the first listed parent, enabling all parent constructors/methods to be called properly if each uses `super()` consistently.
- Edge case: Python raises a **TypeError** at class creation time if a **consistent MRO cannot be computed** (contradictory ordering across the hierarchy), preventing ambiguous/unsafe designs from even being defined.

```

class Flyable:
    def move(self): print("Flying")
class Swimmable:
    def move(self): print("Swimming")
class Duck(Flyable, Swimmable):    # multiple inheritance
    pass

Duck().move()    # "Flying" -- Flyable comes first in MRO
print(Duck.__mro__)

```

## Q51. What is Polymorphism? — real-world example

- **Polymorphism** (“many forms”) is the OOP principle where the **same method name/interface behaves differently** depending on the **object/context** calling it.
- Python supports polymorphism through **duck typing** (“if it walks like a duck and quacks like a duck...”), **method overriding**, and **operator overloading**.

- Real-world example: pressing the “**start**” button behaves differently on a **car** (starts engine), **washing machine** (starts wash cycle), or **computer** (boots OS) — same action name, different implementation per object.
- Types of polymorphism in Python: **Compile-time (Overloading)** — simulated via default args/ `*args` / `multipledispatch` ; **Runtime (Overriding)** — achieved via inheritance and method overriding.
- Polymorphism lets you write **generic code** that works with objects of different classes as long as they support a common interface/method — improves **flexibility and extensibility**.
- Built-in example: the `len()` function and `+` operator behave differently for strings, lists, and custom objects — this is polymorphism at the language level.

```
class Cat:
    def sound(self): print("Meow")
class Dog:
    def sound(self): print("Woof")

for animal in [Cat(), Dog()]:
    animal.sound()    # same method call, different behavior -> polymorphism
```

## Q52. What is Operator Overloading in Python? — real-world example

- **Operator overloading** allows **built-in operators** (`+`, `-`, `*`, `==`, `<`, etc.) to work with **user-defined objects** by defining the corresponding **magic/dunder methods** in the class.
- Achieved by implementing methods like `__add__` (for `+`), `__sub__` (for `-`), `__eq__` (for `==`), `__lt__` (for `<`), `__mul__` (for `*`), etc.
- Real-world example: adding two **complex numbers** (`3+2j + 1+4j`) or two **vectors** (`v1 + v2`) — the `+` operator is “overloaded” to mean vector/complex addition instead of simple numeric addition.
- It makes custom objects behave **intuitively**, like built-in types, improving **code readability** (`p1 + p2` instead of `p1.add(p2)`).
- Python doesn’t allow creating brand-new operators, but existing ones can be **redefined per class** — this is different from arithmetic on unrelated types, which would raise `TypeError` unless overloaded.
- Edge case: if `__radd__` (reflected add) is not defined, `int + CustomObject` may fail even if `CustomObject + int` works — order matters unless both magic methods are defined.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):          # overloads '+'
        return Vector(self.x + other.x, self.y + other.y)
    def __str__(self):
        return f"({self.x}, {self.y})"

v1, v2 = Vector(1, 2), Vector(3, 4)
print(v1 + v2)    # (4, 6)
```

## Q53. What is Method Overloading in Python? — real-world example

- **Method overloading** (in traditional OOP languages like Java/C++) means defining **multiple methods with the same name but different parameter lists** (count/type) within the same class.
- **Python does NOT support true method overloading** — if you define the same method name twice, the **second definition simply overwrites the first** (edge case/common interview trap).
- Real-world example (conceptually): a `print()` operation that could work with different inputs — one number, two numbers, or a string — behaving appropriately depending on what’s passed.
- Python **simulates** overloading using: **default arguments**, **variable-length arguments** (`*args`, `**kwargs`), or the `multipledispatch` third-party module (`@dispatch`) for true type-based overloading.
- Using default/variable arguments is the **Pythonic** way; `multipledispatch` is used when strict type-based overload resolution is required, similar to statically typed languages.
- This is a key **edge case** to mention in interviews: Python’s dynamic typing philosophy makes traditional overloading unnecessary/redundant in most cases — a single flexible method usually suffices.

```
# Native attempt -- does NOT achieve true overloading
class Demo:
```

```

def show(self, a):
    print(a)
def show(self, a, b):      # this OVERWRITES the previous show()
    print(a, b)
# Demo().show(5)  # TypeError: missing 1 required positional argument

# Pythonic simulation using default args
class Calculator:
    def add(self, a, b=0, c=0):
        return a + b + c

print(Calculator().add(5))
print(Calculator().add(5, 10))
print(Calculator().add(5, 10, 15))

```

## Q54. What is Method Overriding in Python? — real-world example

- **Method overriding** occurs when a **child (subclass)** redefines a **method** that is already defined in its **parent class**, with the **same name and signature**, to provide a **specialized implementation**.
- It is a form of **runtime polymorphism** — the method that actually executes is determined by the **object's actual type**, not the reference type, at runtime.
- Real-world example: a generic `Employee.calculate_bonus()` method overridden differently in `Manager` and `Developer` subclasses to reflect different bonus policies.
- The overriding method can still call the parent's original version using `super().method_name()`, allowing it to **extend** rather than completely replace the parent behavior.
- Overriding requires **inheritance** (unlike overloading, which is about multiple methods within the same class) — this is a key distinguishing point in interviews.
- Edge case: if the overriding method's signature differs significantly (different required parameters), calling it polymorphically through a general reference/list can lead to **runtime errors** — Python does not check signature compatibility at override time (no compile-time enforcement).

```

class Employee:
    def calculate_bonus(self):
        return 1000

class Manager(Employee):
    def calculate_bonus(self):      # overriding
        base = super().calculate_bonus()  # extend parent logic
        return base + 5000

print(Manager().calculate_bonus())  # 6000

```

## Q55. What is the `multipledispatch` module and why do we need to install it?

- `multipledispatch` is a **third-party Python package** that enables **true method/function overloading based on argument types (and count)** — something core Python does not support natively.
- It provides the `@dispatch(type1, type2, ...)` decorator; you define multiple versions of a function/method with the **same name** but different `@dispatch` type signatures, and the correct version is chosen automatically **at runtime based on argument types**.
- We need to **install it separately** using `pip install multipledispatch`, because it is **not part of the Python standard library**.
- It's useful when writing **type-sensitive APIs** where behavior should genuinely differ based on argument types (e.g., `add(int, int)` vs `add(str, str)` vs `add(list, list)`).
- Internally, it maintains a **dispatch table/registry** mapping type signatures to function implementations, and picks the **best match** when the function is called.
- Edge case: if no matching type signature is found for the given arguments, it raises a `NotImplementedError` — unlike default Python behavior which would just fail with a generic `TypeError` on the wrong overwritten function.

```
pip install multipledispatch
```

```
from multipledispatch import dispatch
```

```

class Calculator:
    @dispatch(int, int)
    def add(self, a, b):
        return a + b

    @dispatch(str, str)
    def add(self, a, b):
        return a + " " + b

c = Calculator()
print(c.add(2, 3))           # 5
print(c.add("Hello", "World")) # Hello World

```

## Q56. What is the PIP command and how does it work?

- **PIP** stands for “**Pip Installs Packages**” (recursive acronym) — it is Python’s **standard package manager**, used to install, upgrade, and manage third-party libraries/packages from **PyPI (Python Package Index)**.
- Common commands: `pip install <package>` (install), `pip uninstall <package>` (remove), `pip list` (list installed packages), `pip show <package>` (show details), `pip freeze > requirements.txt` (export dependencies).
- It works by **connecting to PyPI (or a specified index/repository)**, downloading the package’s **wheel/source distribution**, and installing it into the Python environment’s `site-packages` directory.
- Supports **version control** — e.g., `pip install package==1.2.0` installs a specific version, `pip install --upgrade package` upgrades to the latest.
- Works closely with **virtual environments** (`venv`, `virtualenv`) to keep project-specific dependencies isolated from the global Python installation.
- Essential for OOP-related third-party tools mentioned earlier, e.g., `pip install multipledispatch` — without pip, using such external modules would require manual download/setup.

```

pip install multipledispatch # install
pip show multipledispatch   # check details
pip uninstall multipledispatch # remove
pip list                     # list all installed packages

```

## Q57. Differences between Method Overloading and Method Overriding in Python

| Method Overloading                                                              | Method Overriding                                                        |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Same method name, different parameters, <b>same class</b>                       | Same method name & signature, defined in <b>parent and child</b> classes |
| Not natively supported in Python                                                | Fully supported natively in Python                                       |
| Simulated via default args / <code>*args</code> / <code>multipledispatch</code> | Achieved via inheritance                                                 |
| Resolved conceptually at “ <b>compile time</b> ” in languages that support it   | Resolved at <b>runtime</b> , based on actual object type                 |
| Does not require inheritance                                                    | Requires inheritance (parent-child relationship)                         |
| Represents <b>compile-time polymorphism</b>                                     | Represents <b>runtime polymorphism</b>                                   |

- Overloading is about having **multiple variants of behavior within one class**; overriding is about **redefining inherited behavior across classes**.
- Python’s dynamic typing and default/variable arguments make native overloading largely unnecessary; overriding, in contrast, is a **core, fully-supported OOP feature**.
- Overriding requires the **exact same method name** (Python doesn’t check parameter signatures at override time — it simply replaces the method in the subclass’s namespace).
- `super()` is relevant to overriding (to call the parent’s version) but **irrelevant to overloading**.
- Overloading in Python (native) actually results in the **last definition silently replacing earlier ones** (an important “gotcha” to state in interviews).
- Both are forms of **polymorphism** but differ in *when* the correct method is determined and *where* the methods are defined.

## Q58. Use of `super()` function in Python — how it works

- `super()` is a **built-in function** used to call methods (commonly the constructor `__init__`) of a **parent/super class** from within a child class.
- It allows a subclass to **reuse and extend** parent behavior instead of completely rewriting it — following the **DRY (Don't Repeat Yourself)** principle.
- In **single inheritance**, `super()` simply refers to the immediate parent class.
- In **multiple inheritance**, `super()` does **not** just mean “the first parent” — it follows the class's computed **MRO (Method Resolution Order)**, calling the **next class in that chain** (this enables cooperative multiple inheritance).
- Common usage: `super().__init__(args)` inside a child's `__init__` to initialize inherited attributes defined in the parent constructor.
- Can also be used to call **any parent method**, not just the constructor — e.g., `super().method_name()` to extend an overridden method's behavior.

```
class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def __init__(self, name, salary):
        super().__init__(name)    # calls Person's __init__
        self.salary = salary

e = Employee("Ravi", 50000)
print(e.name, e.salary)
```

## Q59. Differences between `self` and `super()` in Python

| <code>self</code>                                                    | <code>super()</code>                                              |
|----------------------------------------------------------------------|-------------------------------------------------------------------|
| Refers to the <b>current object instance</b>                         | Refers to the <b>next class in the MRO (typically the parent)</b> |
| Used to access the current object's own attributes/methods           | Used to access parent/ancestor class's attributes/methods         |
| Must be explicitly passed as the first parameter in instance methods | Called as a function; no explicit parameter needed                |
| Available in every instance method                                   | Typically used inside overridden methods/constructors             |
| Points to the object actually calling the method                     | Points to a class higher up in the inheritance chain (per MRO)    |
| Not a keyword — just a naming convention                             | Built-in function                                                 |

- `self` gives access to “**this object's own version**” of data/methods (including overridden ones); `super()` explicitly reaches **up the hierarchy** to the parent/ancestor's version.
- `self.method()` uses **dynamic dispatch based on the object's actual (most-derived) class** — even if called from a parent class method, it resolves to the subclass's override if one exists.
- `super().method()` **deliberately bypasses** the current class's own override and goes to the **next class in MRO**, useful for extending rather than replacing behavior.
- Both are essential for correct behavior in **inheritance and polymorphism** — `self` for object-specific access, `super()` for controlled access to inherited behavior.
- In multiple inheritance, `self.method()` could invoke different code than `super().method()` inside the same class, since `self` uses the *actual* object's MRO-resolved most-derived method, while `super()` explicitly skips to the next class.
- Misusing `self` where `super()` was intended can cause **infinite recursion** if a method calls `self.method()` inside its own override instead of `super().method()`.

```
class A:
    def greet(self): print("Hello from A")

class B(A):
    def greet(self):
        super().greet()    # calls A's greet -- uses super()
        print("Hello from B")
```

```
B().greet()
```

## Q60. What is an Error in Python? Different types of Errors

- An **error** is a problem in a program that **prevents it from running/executing correctly**, often detected before or during execution, typically indicating a flaw in code structure or logic.
- Python errors are broadly classified into: **Syntax Errors** (parsing errors — invalid grammar, detected before execution) and **Runtime Errors** (also called exceptions — occur during execution, e.g., division by zero).
- **Syntax Error**: caused by incorrect Python syntax (missing colon, unmatched brackets) — the code **won't even start running**.
- **Logical Error**: the program runs without crashing but produces an **incorrect result**, due to a mistake in logic (Python doesn't detect these automatically — hardest to debug).
- **Runtime Error (Exception)**: occurs **while the program is executing**, e.g., `ZeroDivisionError`, `FileNotFoundError`, `IndexError` — these are the ones handled via `try-except`.
- Understanding error types helps decide the debugging/handling strategy: syntax errors must be **fixed in code**; runtime errors can be **handled gracefully** using exception handling.

```
# Syntax Error example
# def show()    print("hi")    -> missing colon, invalid syntax

# Runtime Error (Exception) example
print(10 / 0)    # ZeroDivisionError
```

## Q61. What is an Exception in Python?

- An **exception** is an **event/object** that disrupts the **normal flow of program execution**, raised automatically when Python encounters a runtime error, or manually via the `raise` keyword.
- In Python, exceptions are **objects** — all built-in exceptions derive from the base class `BaseException`, with most user-relevant ones deriving from `Exception`.
- When an exception occurs and is **not handled**, the program **terminates abnormally**, printing a **traceback**.
- Exceptions can be **caught and handled gracefully** using `try-except` blocks, allowing the program to **continue execution** instead of crashing.
- Common built-in exceptions: `ZeroDivisionError`, `TypeError`, `ValueError`, `IndexError`, `KeyError`, `FileNotFoundError`, `AttributeError`.
- Python also allows defining **custom (user-defined) exceptions** by subclassing `Exception`, useful for domain-specific error handling (see Q68).

```
try:
    x = 10 / 0
except ZeroDivisionError as e:
    print("Exception caught:", e)
```

## Q62. Differences between Error and Exception in Python

| Error                                                              | Exception                                                                                   |
|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Broader term — includes syntax & logical issues                    | A specific runtime event, a subclass of <code>BaseException</code> / <code>Exception</code> |
| Syntax errors are usually <b>not recoverable</b> /handleable       | Exceptions <b>can be caught and handled</b> using try-except                                |
| Detected often before execution (syntax) or as logic flaws         | Detected/raised <b>during</b> program execution                                             |
| Not always represented as a Python object                          | Always represented as an <b>exception object</b> with a class hierarchy                     |
| Cannot be "caught" via <code>try-except</code> (for syntax errors) | Can be caught, logged, and program flow continued                                           |
| Indicates something fundamentally wrong with code structure/logic  | Indicates an abnormal but often anticipated/handle-able runtime condition                   |

- All exceptions are technically a **type of runtime error**, but not all errors are exceptions (e.g., `SyntaxError`, though ironically represented as a class too, occurs **before** execution and generally can't be "handled" mid-run the way runtime exceptions can).
- Exceptions provide a **structured, object-oriented way** to represent and respond to runtime problems, unlike generic/logical errors.
- Exception handling ( `try-except` ) is specifically designed to deal with **exceptions**, not with syntax errors (which must be fixed in the source code itself).
- In Python's class hierarchy, `SyntaxError` is actually **also a subclass of `Exception`**, but practically it's raised at **parse time**, before your `try-except` blocks even get a chance to run (important nuance/edge case).

## Q63. Important types of Errors — with examples

- `SyntaxError`: invalid Python grammar.

```
# if True # missing colon -> SyntaxError
```

- `IndentationError`: incorrect/inconsistent indentation (subclass of `SyntaxError`).

```
# def f():
# print("hi") # IndentationError: expected an indented block
```

- `NameError`: using a variable/name that hasn't been defined.

```
print(undefined_var) # NameError
```

- `TypeError`: an operation applied to an inappropriate/incompatible type.

```
print("5" + 5) # TypeError: can only concatenate str
```

- `ValueError`: correct type but inappropriate/invalid value.

```
int("abc") # ValueError
```

- `RecursionError`: exceeding the maximum recursion depth (infinite/too-deep recursion).

```
def f(): return f()
# f() # RecursionError
```

## Q64. Important types of Exceptions — with examples

- `ZeroDivisionError`: raised when dividing a number by zero.

```
print(10 / 0) # ZeroDivisionError
```

- `IndexError`: raised when accessing a sequence (list/tuple/string) index that's **out of range**.

```
lst = [1, 2, 3]
print(lst[5]) # IndexError
```

- `KeyError`: raised when accessing a **non-existent key** in a dictionary.

```
d = {"a": 1}
print(d["b"]) # KeyError
```

- `FileNotFoundError`: raised when trying to open a file that **doesn't exist**.

```
open("no_such_file.txt") # FileNotFoundError
```

- `AttributeError`: raised when accessing an attribute/method that **doesn't exist** on an object.

```
"hello".unknown_method() # AttributeError
```

- `ImportError` / `ModuleNotFoundError`: raised when a module/import fails to be found or loaded.

```
import non_existing_module # ModuleNotFoundError
```

## Q65. What is Exception Handling in Python?

- **Exception handling** is the mechanism to **detect and respond to runtime errors gracefully**, preventing abrupt program termination.
- Implemented using the `try - except - else - finally` block structure, along with the `raise` keyword for manually triggering exceptions.
- Code that **might fail** is placed inside `try`; the corresponding recovery/handling logic is placed inside one or more `except` blocks.
- Multiple `except` blocks can handle **different exception types differently**, and a generic `except Exception` can catch anything not explicitly handled (should be used cautiously — too broad).
- Exception handling improves **program robustness/resilience**, provides better **user experience** (friendly error messages instead of crashes), and helps in **debugging** via meaningful error logging.
- It follows the principle: **“handle only what you can meaningfully recover from”** — don’t blindly catch and suppress every exception, as it can hide real bugs.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Invalid input, enter a number")
else:
    print("Result:", result)
finally:
    print("Execution complete")
```

## Q66. `try`, `except`, `except-else`, `finally`, and `raise` keywords — with example programs

- `try`: the block of code that might raise an exception; Python monitors it for errors.
- `except`: catches and handles a **specific (or general) exception type** raised inside the `try` block; multiple `except` blocks can target different error types.
- `else`: executes **only if the `try` block completes without any exception** — used for code that should run only on success (keeps it separate from error-prone code).
- `finally`: always executes, **whether or not an exception occurred** — typically used for **cleanup** (closing files, releasing resources, closing DB connections).
- `raise`: used to **manually trigger** an exception — either a built-in one or a custom exception — optionally with a custom message.
- Execution order: `try` → (if error) `except` → (if no error) `else` → `finally` (always last, regardless of path taken).

```
def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: division by zero")
        raise # re-raise the same exception
    else:
        print("Division successful")
        return result
    finally:
        print("Cleanup: divide() finished")

try:
    divide(10, 0)
except ZeroDivisionError:
```

```
print("Handled in caller")
```

## Q67. The `finally` clause — advantages, with real-world example

- The `finally` block contains code that is **guaranteed to execute**, regardless of whether an exception was raised, caught, or even if a `return` / `break` / `continue` occurs inside the `try` / `except`.
- Its primary purpose is **resource cleanup** — closing files, releasing network/database connections, releasing locks — ensuring resources aren't left open/leaked even if an error occurs.
- Real-world example: opening a **database connection** to fetch data — even if a query fails (exception raised), the connection **must be closed** in `finally` to avoid connection leaks.
- **Advantage:** guarantees **deterministic cleanup**, unlike relying on code placed after the `try` block (which would be **skipped** if an unhandled exception propagates out).
- `finally` runs even if the `except` block itself raises a **new exception**, or if there's a `return` statement inside `try` / `except` — a subtle but important edge case.
- Best practice in modern Python: for file/resource handling, the **with statement (context manager)** is often preferred over manual `try-finally`, since it automatically manages cleanup via `__enter__` / `__exit__`.

```
def read_file(path):
    f = None
    try:
        f = open(path, "r")
        return f.read()
    except FileNotFoundError:
        print("File not found")
    finally:
        if f:
            f.close()          # guaranteed cleanup
            print("Attempted file close")
```

## Q68. User-defined Exceptions — how to create them in Python

- A **user-defined (custom) exception** is created by defining a **new class that inherits from `Exception`** (or one of its built-in subclasses), allowing you to represent **domain-specific error conditions**.
- Custom exceptions improve **code clarity** — instead of a generic `ValueError`, you can raise `InsufficientBalanceError`, immediately conveying the exact business problem.
- You can override `__init__` in the custom exception class to accept and store **additional context/data** (e.g., the invalid value, an error code) beyond just a message.
- Custom exception classes are typically named ending with **"Error"** by convention, and can form their own **hierarchy** (e.g., `AppError` as a base, with multiple specific subclasses).
- They are raised using the `raise` keyword, just like built-in exceptions, and caught using `except CustomException as e:`.
- Allows **fine-grained exception handling** — callers can choose to catch only your specific custom exception, or a broader built-in `Exception`, giving flexible error-handling granularity.

```
class InsufficientBalanceError(Exception):    # custom exception
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"Cannot withdraw {amount}, balance is only {balance}")

class BankAccount:
    def __init__(self, balance):
        self.balance = balance
    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientBalanceError(self.balance, amount)
        self.balance -= amount

try:
    BankAccount(500).withdraw(1000)
except InsufficientBalanceError as e:
    print("Error:", e)
```

## Q69. Importance of the `raise` keyword in Exception Handling

- `raise` is used to **manually/explicitly trigger an exception**, either a built-in one or a custom user-defined one, at any point in the code.
- It allows developers to enforce **business rules/validation** — e.g., raising `ValueError` if an invalid argument is passed, even though no “natural” Python error would occur otherwise.
- `raise` can be used **without arguments inside an `except` block** to **re-raise the currently handled exception**, preserving its original traceback — useful for logging an error before letting it propagate further.
- Supports **exception chaining** using `raise NewException(...) from original_exception`, which preserves the **root cause** for better debugging (shows “The above exception was the direct cause of...”).
- It's essential for implementing **custom exceptions** meaningfully — defining a custom exception class is useless without actually `raise`-ing it somewhere in the logic.
- Using `raise` appropriately (instead of silently returning `None` /error codes) follows Python's “**Errors should never pass silently**” philosophy (from the Zen of Python), making bugs easier to detect early.

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")    # explicit raise
    return age

try:
    set_age(-5)
except ValueError as e:
    print("Caught:", e)
```

## Q70. Exception Propagation in Python

- **Exception propagation** refers to how an exception, if **not caught** in the function where it occurs, **travels up the call stack** to the calling function, and further up, until it is either caught by some `except` block or reaches the top level (causing the program to terminate).
- If `function_A()` calls `function_B()`, and `function_B()` raises an exception without handling it, the exception **propagates back to `function_A()`** — if `function_A()` also doesn't handle it, it propagates further to whoever called `function_A()`, and so on.
- If the exception is **never caught anywhere** in the call chain, Python's default handler catches it at the top, prints a **full traceback**, and terminates the program.
- Propagation allows handling to happen at the **most appropriate level** — e.g., a low-level function might not know how to recover from an error, but a higher-level caller (closer to the user) does, so it's fine to let it propagate up.
- The **traceback** shown on an uncaught exception reflects the **entire propagation path** — from where it was raised, through every calling function, up to the top level (very useful for debugging).
- You can **stop propagation early** at any level using `try-except`, and can also **deliberately re-propagate** (using `raise` with no arguments) after performing some local action like logging.

```
def level3():
    raise ValueError("Something went wrong in level3")

def level2():
    level3()                # exception not handled here -> propagates up

def level1():
    try:
        level2()           # exception not handled in level2 -> propagates here
    except ValueError as e:
        print("Caught at level1:", e)    # finally handled here

level1()
```

## Quick Revision Summary Table (Key OOP Keywords)

| Concept | Python Keyword/Tool | Key Point to Remember              |
|---------|---------------------|------------------------------------|
|         |                     | Bundling data + methods; restricts |

|                    |                                                                |                                             |
|--------------------|----------------------------------------------------------------|---------------------------------------------|
| Encapsulation      | <code>_var</code> , <code>__var</code>                         | access                                      |
| Abstraction        | <code>abc</code> , <code>@abstractmethod</code>                | Hides “how”, exposes “what”                 |
| Inheritance        | <code>class Child(Parent):</code>                              | Code reuse; “is-a” relationship             |
| Polymorphism       | Method Overriding, Duck Typing                                 | Same interface, different behavior          |
| Constructor        | <code>__init__()</code>                                        | Auto-called; initializes object             |
| self               | first param of instance methods                                | Refers to current object                    |
| super()            | <code>super().__init__()</code>                                | Calls next class in MRO                     |
| MRO                | <code>Class.__mro__</code> , C3 Linearization                  | Order of method lookup in multi-inheritance |
| Method Overloading | Not native; <code>*args</code> , <code>multipledispatch</code> | Last definition overwrites in native Python |
| Method Overriding  | Redefining in subclass                                         | Runtime polymorphism                        |
| Static Method      | <code>@staticmethod</code>                                     | No self/cls                                 |
| Class Method       | <code>@classmethod</code>                                      | Uses <code>cls</code> , accesses class data |
| Name Mangling      | <code>_var</code> → <code>__ClassName__var</code>              | Avoids naming clashes in inheritance        |
| Exception Handling | <code>try/except/else/finally/raise</code>                     | Graceful error recovery                     |
| Custom Exception   | <code>class MyError(Exception):</code>                         | Domain-specific error reporting             |

*End of document — prepared for interview revision. Practice explaining each answer out loud and running the code examples to reinforce understanding.*